



INDIVIDUAL ASSIGNMENT

SENG24A1

SERVER SIDE PROGRAMMING - II

MODULE CODE: COMP50016

CB Number: CB014241

Name: Diwya Semini Jayawardana

Batch Code: SENG24A1

Q-Less Campus SaaS Platform: Security Architecture and Vulnerability Audit Report

Hosted Link: <https://qlesscampus.infinityfreeapp.com/>

Git Hub Repo Link: <https://github.com/Diwya-Semini/qless-campus-laravel.git>

Table of Contents

1. Introduction	3
2. Core Vulnerabilities of Application and Framework Mitigations.....	3
2.1. SQL Injection(SQLi).....	3
2.2. Cross-Site Request Forgery.....	5
2.3. Broken Access Control (Spoofing)	5
2.4. Race Conditions and Data Corruption	6
2.5. Brute-Force Authentication Fuzzing.....	7
2.6. Cross – Site Scripting Prevention.....	8
2.7. Transport Layer Security (HTTPS).....	8
3. Critical API Functional Testing.....	9
3.1. Test Case 1: Student Authentication (POST /api/login)	9
3.2. Test Case 2: Menu (GET /api/menu)	12
3.3. Test Case 3: Cart Checkout (POST /api/order).....	14
3.4. Test Case 4: Log out form a sanctum session (POST /api/logout)	15
4. Hosting and Live Cloud Development	16
4.1. Production Infrastructure and asset pipeline	16
4.2. Database Management	17

1. Introduction

Q-Less campus is a software as a service (SaaS) and it is a large scale application that helps complex needs of businesses that provides multi-tenant canteen operations, student canteen digital menus and food queue. This platform gives a secure multi-tenant environment, managing different structural context of high level administration, canteen manager and student customers, while preserving secure data isolation and cryptographic protection.

To limit the risk in the student order checkouts and access permissions this application is developed using **Laravel 12** framework which has building security features.

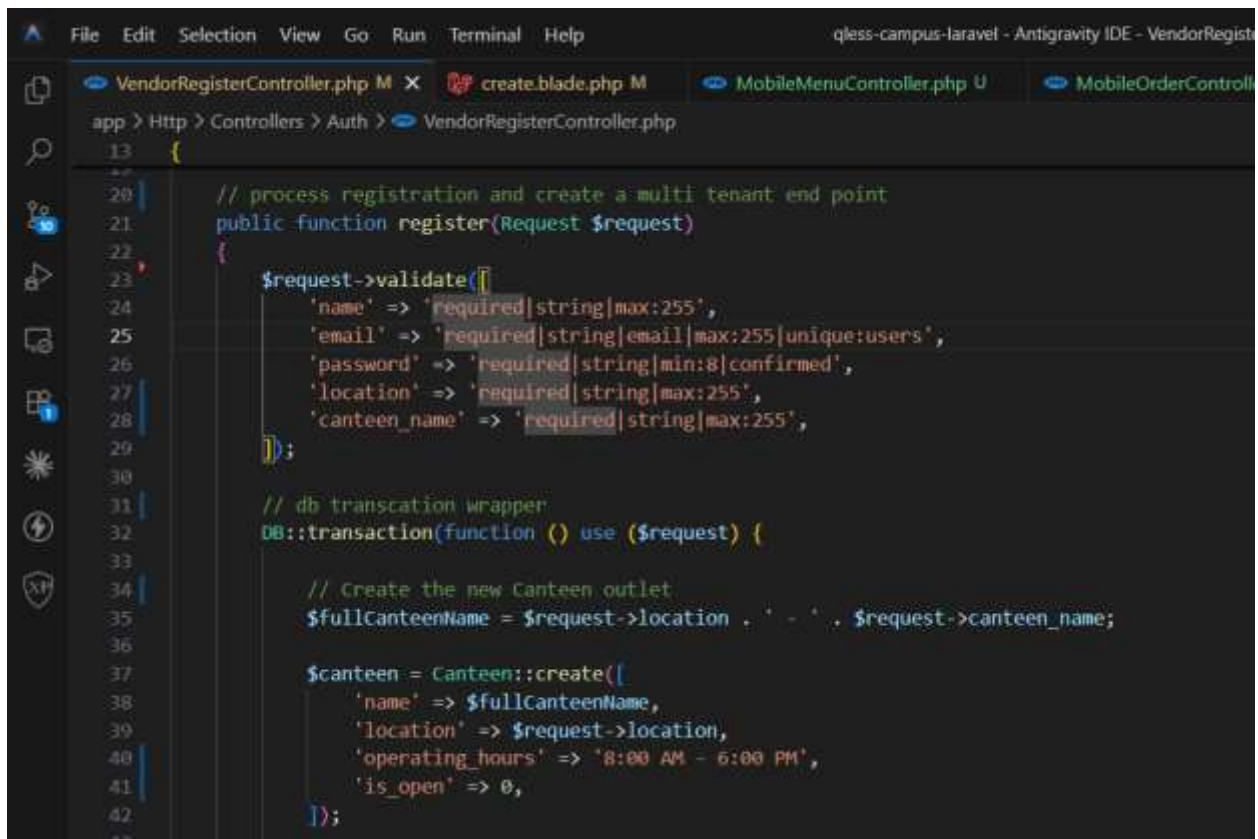
This report is a detail security audit of how the system's defenses against critical web vulnerabilities in OWASP Top 10 framework. Additionally, the documentation describes how laravel implements its build in security like PDO, CSRF Tokens, Automatic Rate Limiting Throttles, and Atomic database transaction look to provide data integrity. And how Laravel's integration with Laravel Sanctum to make a high secure state isolated and scope restricted RESTful API for student mobile pipeline.

2. Core Vulnerabilities of Application and Framework Mitigations.

2.1.SQL Injection(SQLi)

Threat Identification: This happens when a hacker types a database code (SQL Query) into a text box (login page text field). If the system is weak this can fetch the data from the database and show sensitive information, login thought the login page, or deleting tables in database.

Migrate the Threat: My system stops this threat using Laravel's Eloquent ORM to talk with database. Eloquent use PDO (PHP Data Object) this means when a student log in the database treats what they have types as plain text not as code. Even someone types a database query the system ignores that as a text string.

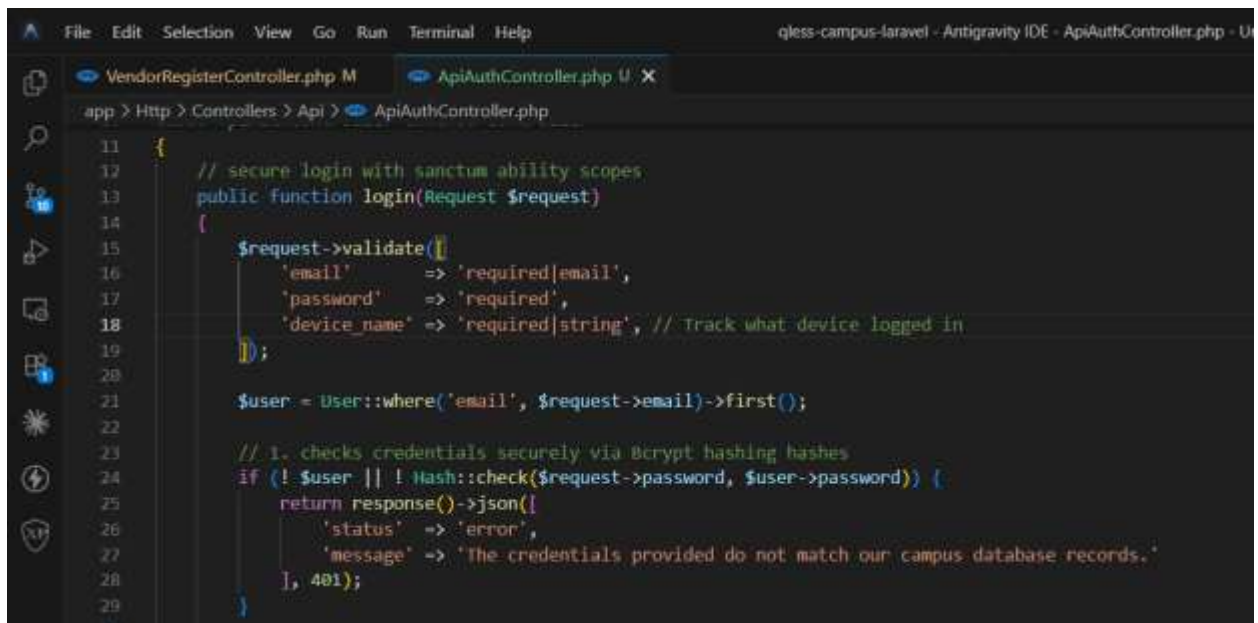


```

13 {
14
15 // process registration and create a multi tenant end point
16 public function register(Request $request)
17 {
18     $request->validate([
19         'name' => 'required|string|max:255',
20         'email' => 'required|string|email|max:255|unique:users',
21         'password' => 'required|string|min:8|confirmed',
22         'location' => 'required|string|max:255',
23         'canteen_name' => 'required|string|max:255',
24     ]);
25
26 // db transaction wrapper
27 DB::transaction(function () use ($request) {
28
29     // Create the new Canteen outlet
30     $fullCanteenName = $request->location . ' - ' . $request->canteen_name;
31
32     $canteen = Canteen::create([
33         'name' => $fullCanteenName,
34         'location' => $request->location,
35         'operating_hours' => '8:00 AM - 6:00 PM',
36         'is_open' => 0,
37     ]);
38 }
39 }

```

Figure 1: Web Registration Controller using Laravel Eloquent models to protect SQL Injection



```

11 {
12 // secure login with sanctum ability scopes
13 public function login(Request $request)
14 {
15     $request->validate([
16         'email' => 'required|email',
17         'password' => 'required',
18         'device_name' => 'required|string', // Track what device logged in
19     ]);
20
21     $user = User::where('email', $request->email)->first();
22
23 // 1. checks credentials securely via Bcrypt hashing hashes
24 if (! $user || ! Hash::check($request->password, $user->password)) {
25     return response()->json([
26         'status' => 'error',
27         'message' => 'The credentials provided do not match our campus database records.'
28     ], 401);
29 }
30 }

```

Figure 2: Mobile Authentication controller utilizing database parameters filtering to block SQL Injection

2.2. Cross-Site Request Forgery

Threat Identification: CSRF happens when an attacker creates a malicious website or link that contains a hidden request that target a site where user is logged in. Example: this can trick a log in canteen manager to deleting menu products by using active browser session.

Mitigate the Threat: for our platform laravel uses a security guard call VerfiyCsrfToken. Every time when a user logged in this creates a random security code. All the forms use @csrf tool which hide this code in the page. When submitting the form laravel check the code. If a tricky web site send a unwanted request the codes won't match and laravel blocks it with 419 Page Expired error.

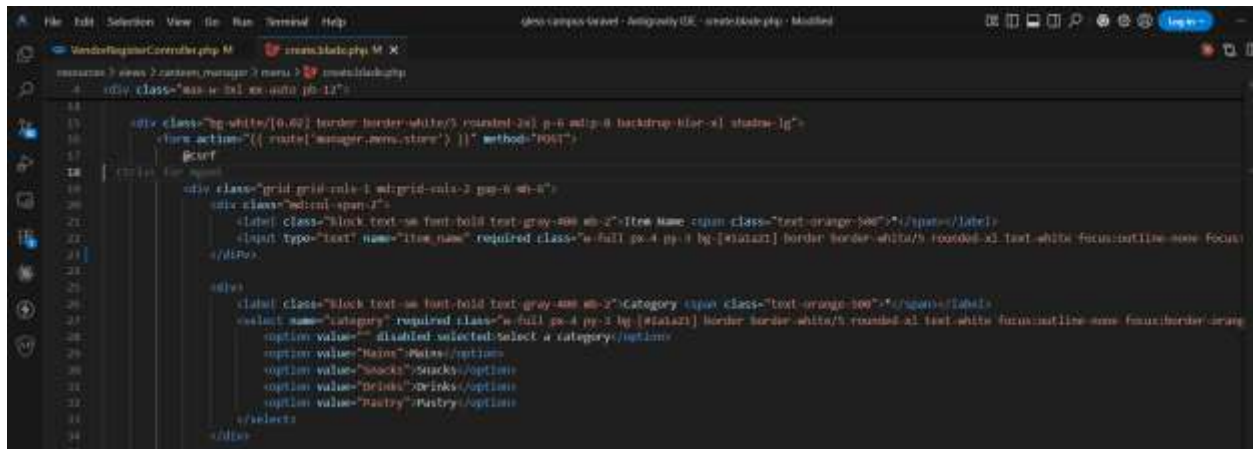
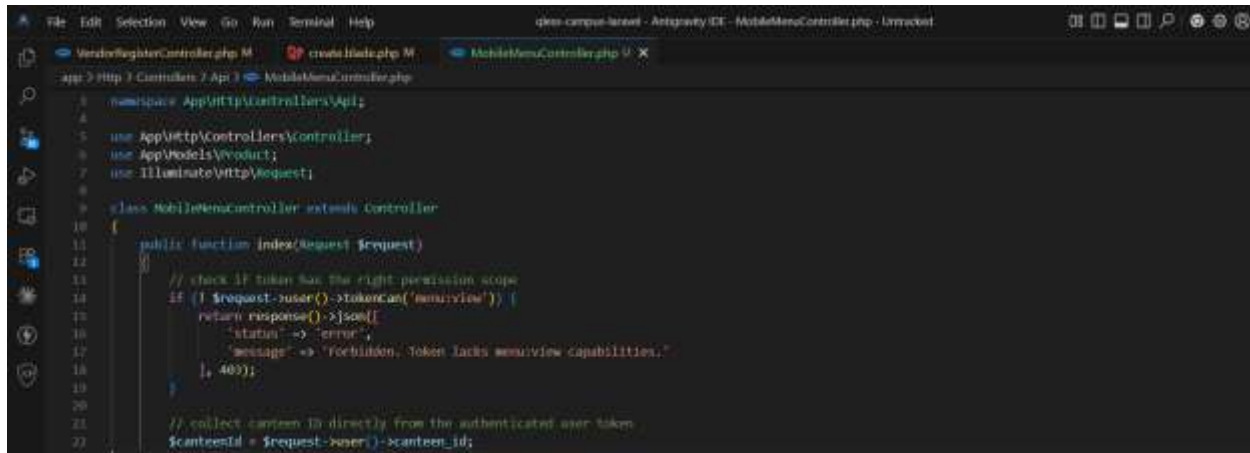


Figure 3: The hidden CSRF token code protecting for new food item creating from

2.3. Broken Access Control (Spoofing)

Threat Identification: This happens when system trust the numbers send form the mobile phone. For example, if the student's phone send canteen_id: 2 or user_id: 1 in normal text an attacker can change those numbers to see different canteen's private information's or make fake orders using a student's user id.

Mitigate the Threat: My system implements a **Strict Data Isolation**. Student's mobile app could never decide or send their canteen_id or user_id. For that our code looks at the secure digital keycard (Sanctum token) store in server: \$request->user()->canteen_id. Also I have use **Token Scopes** like tokenCan ('orders: create'). If a user does not have the exact same digital key the server blocks them with 403 Forbidden error.

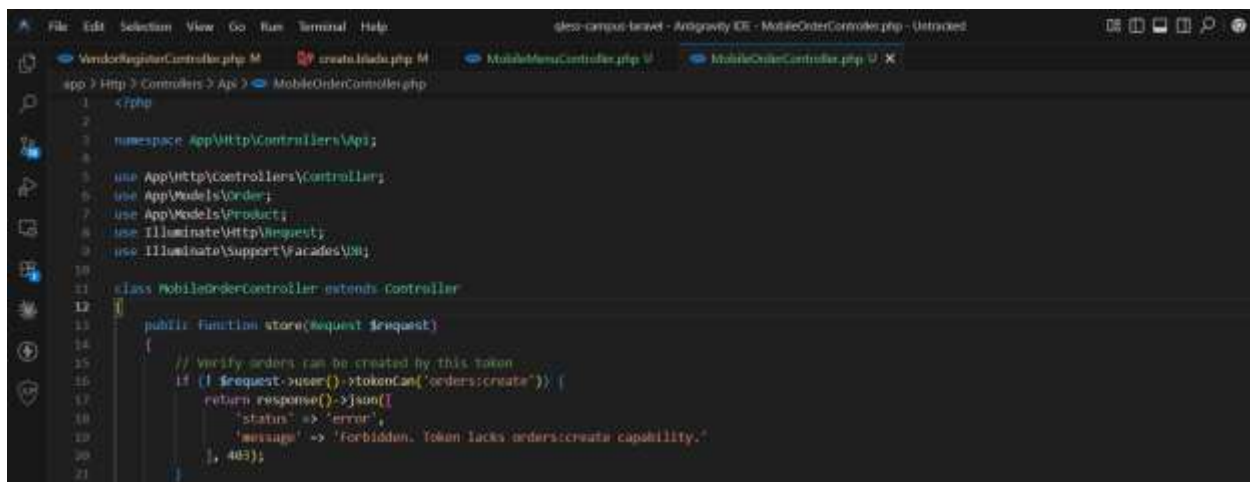


```

1 namespace App\Http\Controllers\Api;
2
3 use App\Http\Controllers\Controller;
4 use App\Models\Product;
5 use Illuminate\Http\Request;
6
7 class MobileMenuController extends Controller
8 {
9     public function index(Request $request)
10     {
11         // check if token has the right permission scope
12         if (! $request->user()->tokenCan('menu:view')) {
13             return response()->json([
14                 'status' => 'error',
15                 'message' => 'Forbidden. Token lacks menu:view capabilities.',
16             ], 403);
17         }
18
19         // collect cart item id directly from the authenticated user token
20         $cartItemId = $request->user()->cartItem_id;
21     }
22 }

```

Figure 4: Figure 4: Using laravel server side token check to stop access control leak - Menu



```

1 <?php
2
3 namespace App\Http\Controllers\Api;
4
5 use App\Http\Controllers\Controller;
6 use App\Models\Order;
7 use App\Models\Product;
8 use Illuminate\Http\Request;
9 use Illuminate\Support\Facades\DB;
10
11 class MobileOrderController extends Controller
12 {
13     public function store(Request $request)
14     {
15         // verify orders can be created by this token
16         if (! $request->user()->tokenCan('orders:create')) {
17             return response()->json([
18                 'status' => 'error',
19                 'message' => 'Forbidden. Token lacks orders:create capability.',
20             ], 403);
21         }
22     }
23 }

```

Figure 5: Using Laravel Server side token checks to stop access control leak – Order

2.4.Race Conditions and Data Corruption

Threat Identification: When a student makes an order the system needs to run multiple queries to create an order, look for prices and save each food items. If the student's phone loose connection halfway through the database transaction an order can be connected without items or with half of items.

Mitigate the Threat: I have use **Atomic Database Transaction** with laravel's DB::beginTransaction(), DB::commit(), and DB::rollback(). Here it has all or nothing rule if any of the steps fail during the process the server catch the error and rollback wipe the data that have being saved.

```

12 {
13     // 1. Database Transaction: Insert order securely to prevent errors
14     DB::beginTransaction();
15     try {
16         // build parent order record
17         $order = Order::create([
18             'user_id' => $user->id,
19             'canteen_id' => $user->canteen_id,
20             'total_amount' => $request->total_amount,
21             'status' => 'pending',
22             'token_id' => rand(1000, 9999),
23         ]);
24
25         // loop and attach specific individual food line items
26         foreach ($request->items as $item) {
27             // pull products price
28             $product = Product::find($item['id']);
29
30             DB::table('order_items')->insert([
31                 'order_id' => $order->id,
32                 'product_id' => $item['id'],
33                 'quantity' => $item['qty'],
34                 'price' => $product->price,
35                 'created_at' => now(),
36                 'updated_at' => now(),
37             ]);
38         }
39
40         DB::commit();
41
42         return response()->json([
43             'status' => 'success',
44             'message' => 'Order placed successfully, sent to kitchen queue.',
45             'order' => [
46                 'id' => $order->id,
47                 'total amount' => $order->total_amount
48             ]
49         ]);
50     } catch (Exception $e) {
51         // Rollback transaction in case of error
52         DB::rollBack();
53         return response()->json([
54             'status' => 'error',
55             'message' => $e->getMessage(),
56         ]);
57     }
58 }

```

Figure 6: Database safeguard for a clean order creating

2.5.Brute-Force Authentication Fuzzing

Threat Identification: A brute force attack is when an attacker uses an automated computer script to find the password by guessing thousand different passwords within a minute until find the correct one.

Mitigate the Threat: Laravel routes mobile login through guard called ThrottleRequests. It counts how many times the IP address try to login by hitting the endpoint. If a computer script guesses the password too many times (ex: more than 60 time a minute), it locks them out and return 429 Too many Requests status code.

2.6. Cross – Site Scripting Prevention

Threat Identification: XSS happens when an attacker injects executable JavaScript script in the input field of the web platform, which are then breaks into another user browser and steal their session tokens.

Mitigate the Threat: I have completely prevented XSS in canteen manager dashboard views. Whenever the web site display database text using double curly brackets ({{ \$variable }}), laravel automatically changes dangerous HTML characters into text characters.

Example: if the attacker type <script> to the inputs laravel converts it into a text string so the code doesn't run.

```

resources > views > canteen_manager > dashboard.blade.php
18 <div class="max-w-7xl mx-auto pb-12">
19
20 <div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6 mb-8">
21
22 <div class="bg-white/[0.02] border border-white/5 rounded-2xl p-6 backdrop-blur-xl shadow-lg relative overflow-hidden">
23 <p class="text-orange-400 text-sm font-bold mt-2 flex items-center">
24
25 </p>
26 </div>
27
28 <!-- live stock availability -->
29 <div class="bg-white/[0.02] border border-white/5 rounded-2xl p-6 backdrop-blur-xl shadow-lg relative overflow-hidden">
30 <p class="text-gray-400 text-sm font-bold tracking-wider uppercase mb-1">Active Menu Items</p>
31 <div class="text-3xl font-black text-white">{{ $activeMenuItems }} <span class="text-gray-500 text-lg font-medium"> / {{ $totalProducts }} </span></div>
32 <p class="({{ $outOfStockItems > 0 ? 'text-red-400' : 'text-green-400' }) text-sm font-bold mt-2 flex items-center">
33 <({{ $outOfStockItems }}) items currently out of stock
34 </p>
35 </div>
36 </div>
37
38 <div class="grid grid-cols-1 lg:grid-cols-2 gap-6">
39
40 <!-- revenue analytics -->
41 <div class="lg:col-span-2 bg-white/[0.02] border border-white/5 rounded-2xl p-6 backdrop-blur-xl shadow-lg h-96 flex flex-col justify-between">
42 <div class="flex justify-between items-center mb-2">

```

2.7. Transport Layer Security (HTTPS)

Threat Identification: When student use public Wi-Fi regular unencrypted internet traffic (HTTP) is send in plain text. A hacker using data sniffing on that Wi-Fi can interrupt the traffic and read students private passwords or Sanctum tokens.

Mitigate the Threat: This web hosting system forces the entire application to use HTTPS encryption. Which wrap all the data going here and there from mobile app to server in a secure cryptographic shield (SSL/TLS). Even hacker's intercepts data in university network it will looks unreadable.

3. Critical API Functional Testing

3.1. Test Case 1: Student Authentication (POST /api/login)

ID	Scenario	Test Steps	Input Data	Expected Output	Status
TC – 1.1	Login with valid credentials	1. POST to /api/login 2. Submit correct credentials	Headers: Accepts: application/json Body: <pre>{ "email": "avishka@slit.lk", "password": "password123", "device_name": "iPhone 15 pro" }</pre>	200 OK Status. And return a secure sanctum token string.	PASS

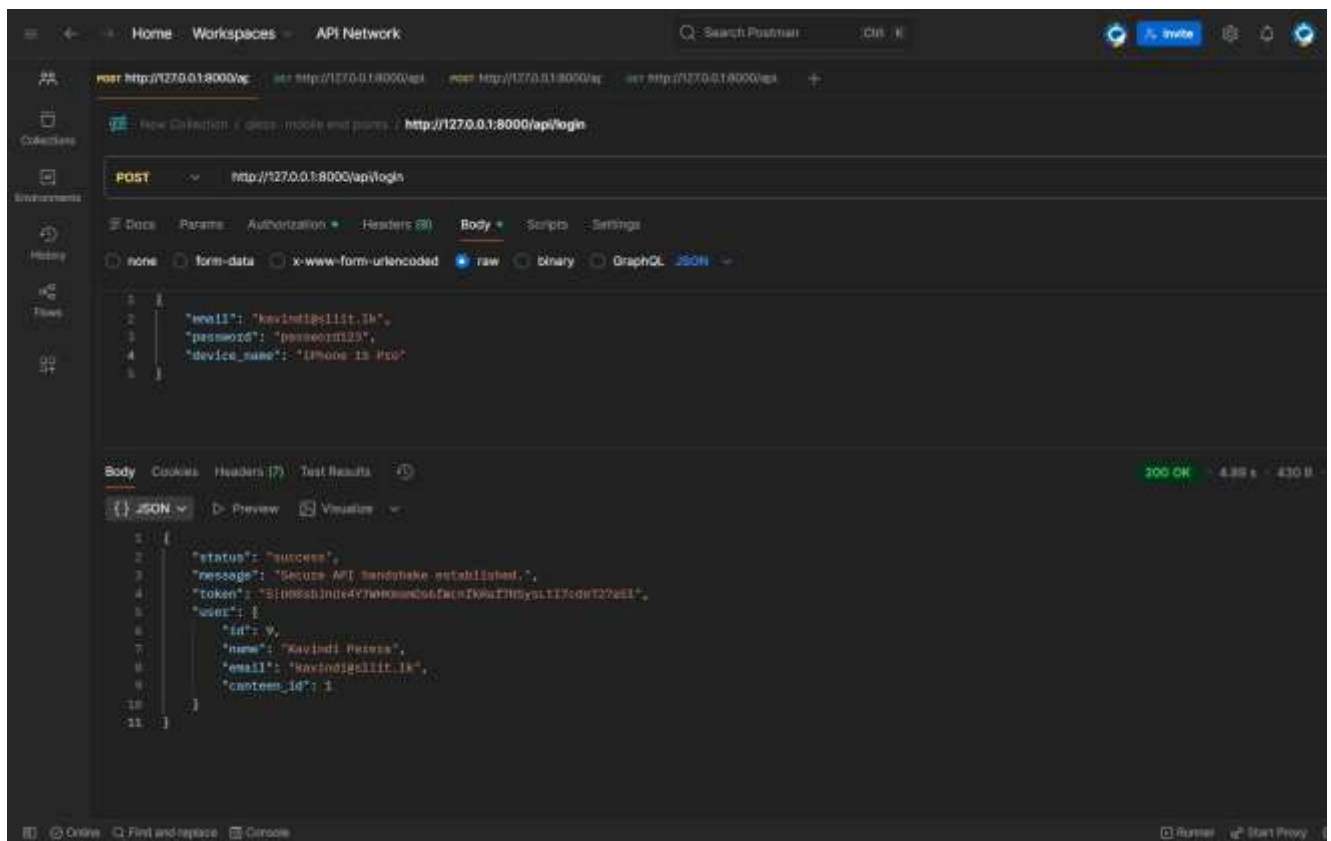


Figure 7: Postman Verification showing successful request and return text Sanctum bearer token

ID	Scenario	Test Steps	Input Data	Expected Output	Status
TC – 1.2	Login without credentials	1. POST to /api/login 2. Submit empty body	Headers: Accepts: application/json Body: {}	422 Unprocessable Content. Server blocks login.	PASS

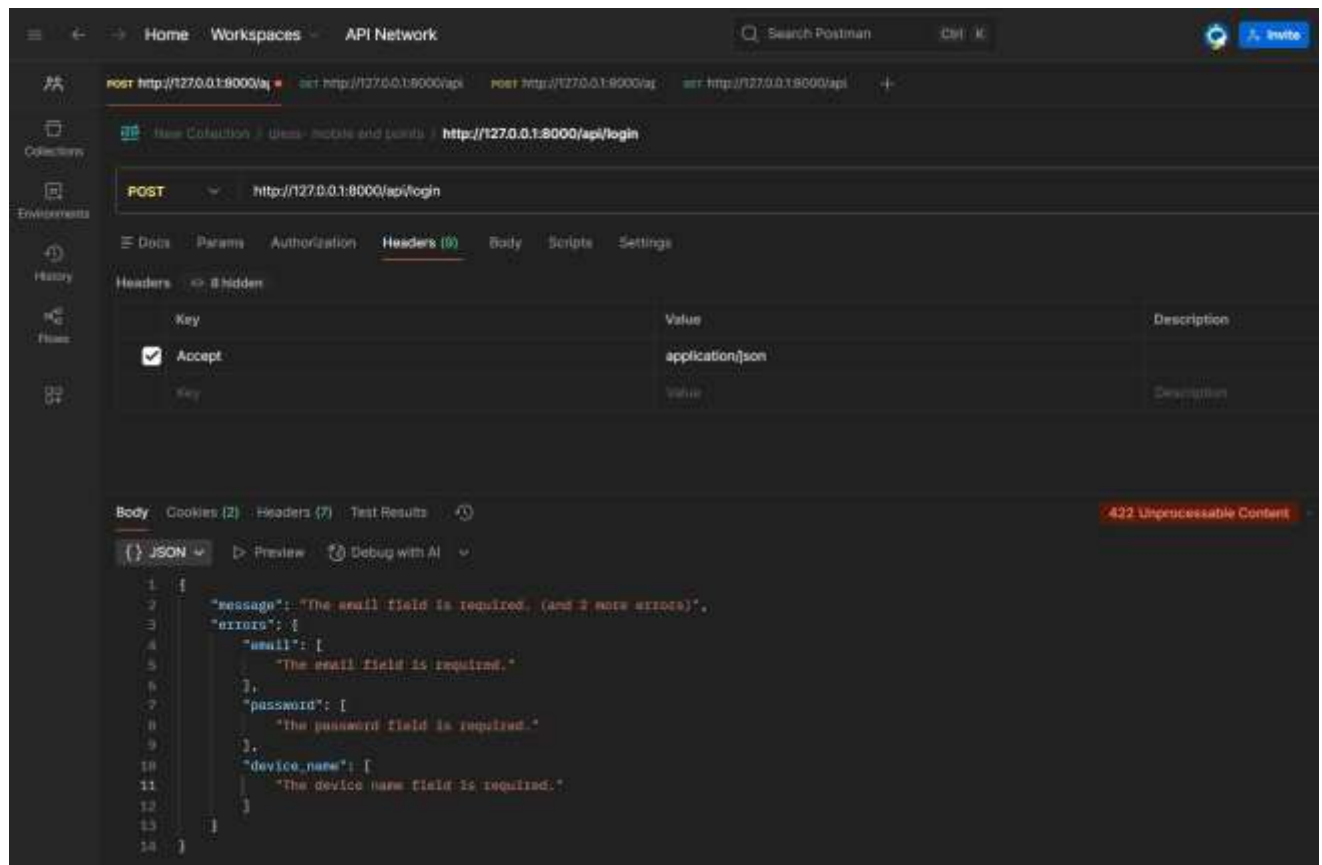


Figure 8: Postman test showing 422 unprocessable content with automatic validation field error.

ID	Scenario	Test Steps	Input Data	Expected Output	Status
TC – 1.3	Login with invalid password	1. POST to /api/login 2. Submit empty body	Headers: Accepts: application/json Body: { "email": "kavindi@sliit.lk", "password": "password456", "device_name": "iPhone 15 Pro" }	401 Unauthenticated Login accept rejecting with error message.	PASS

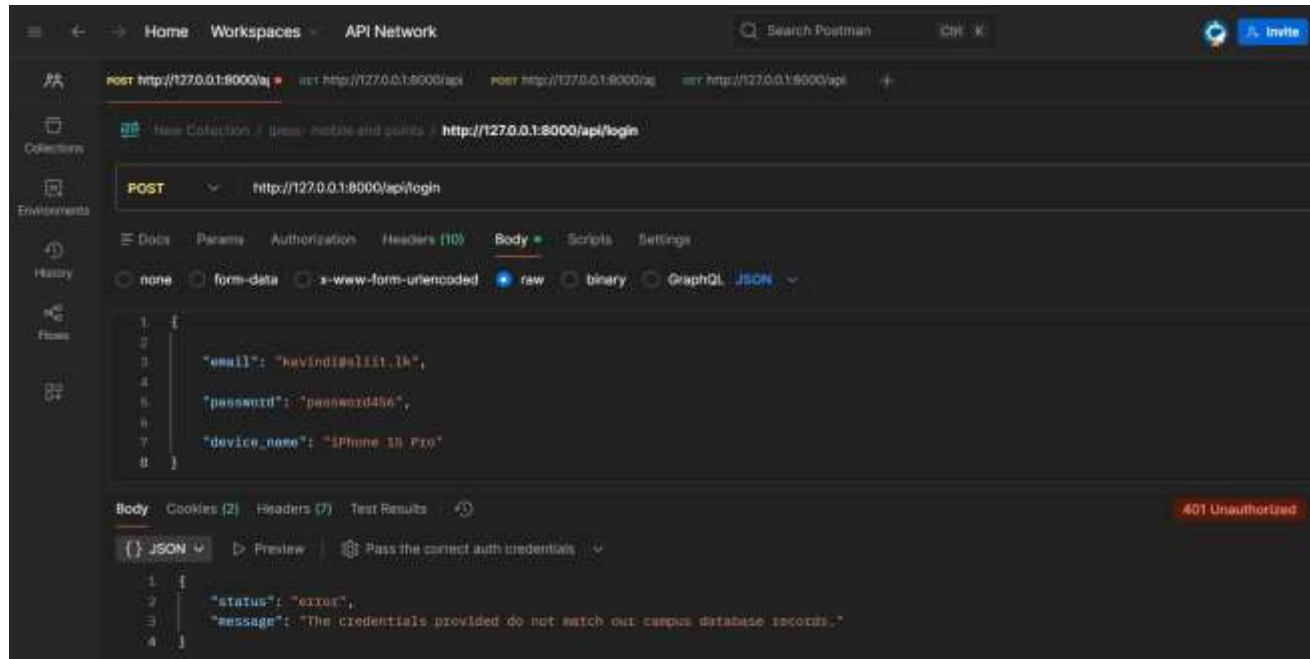


Figure 9: Postman test sending a wrong password returning a 401 Unauthenticated code

3.2. Test Case 2: Menu (GET /api/menu)

ID	Scenario	Test Steps	Input Data	Expected Output	Status
TC – 2.1	Getting Menu details with sanctum token	1. GET to /api/menu 2. Pass active Bearer Token	Headers: Accepts: application/json Authorization: Bearer <valid_token>	200 OK Status.	PASS

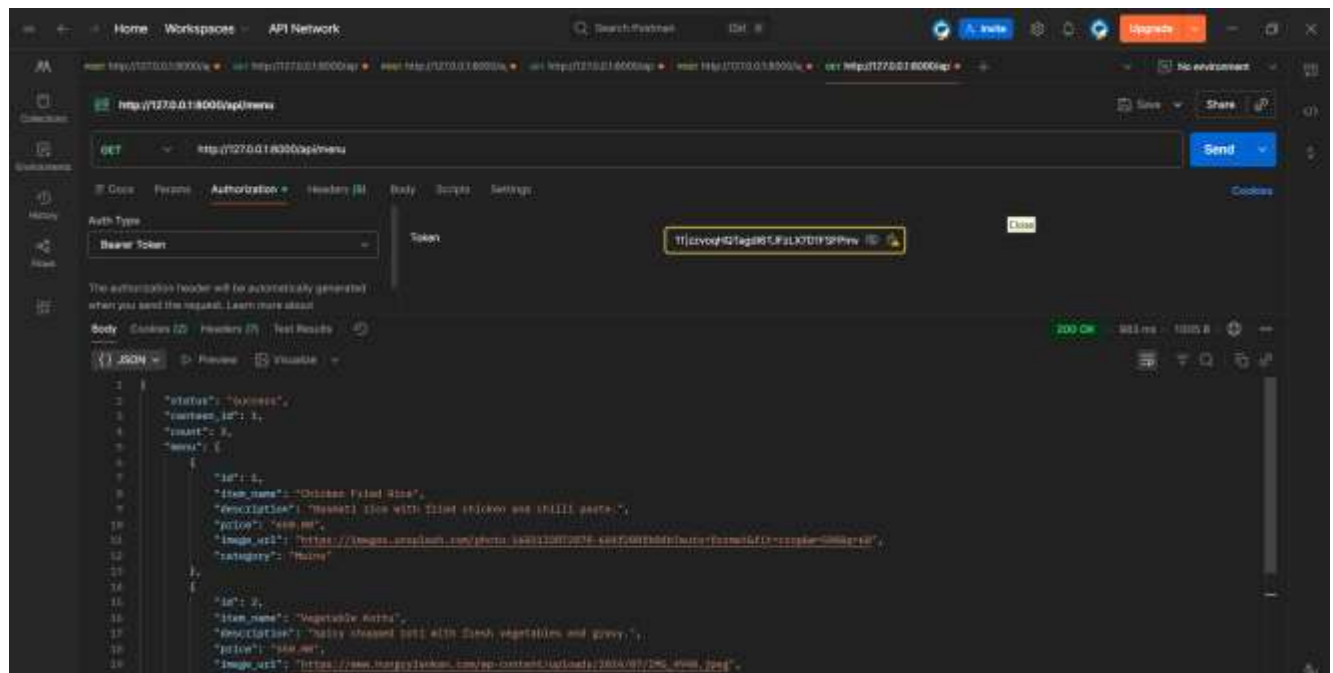


Figure 10: Postman testing (api/menu) with active bearer token

ID	Scenario	Test Steps	Input Data	Expected Output	Status
TC – 2.2	Getting Menu details without sanctum token	1. GET to /api/menu 2. Pass active Bearer Token	Headers: Accepts: application/json Authorization: Blank Bearer <valid_token>	200 OK Status.	PASS

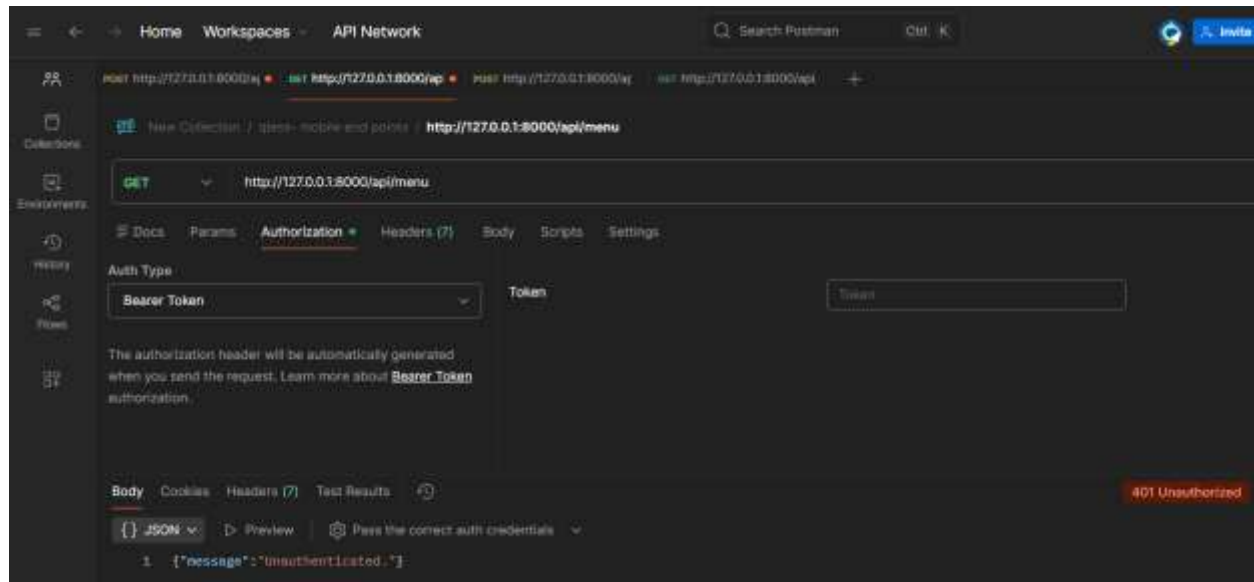


Figure 11: Postman Testing (/api/menu) without the Bearer token

3.3. Test Case 3: Cart Checkout (POST /api/order)

ID	Scenario	Test Steps	Input Data	Expected Output	Status
TC – 3.1	Place an Order with sanctum token	1. POST to /api/orders 2. Submit items cart	Headers: Authorization: Blank Bearer <valid_token> Body: <pre>{ "total_amount": 750.00, "items": [{ "id": 1, "qty": 2 }] }</pre>	200 OK Status.	PASS

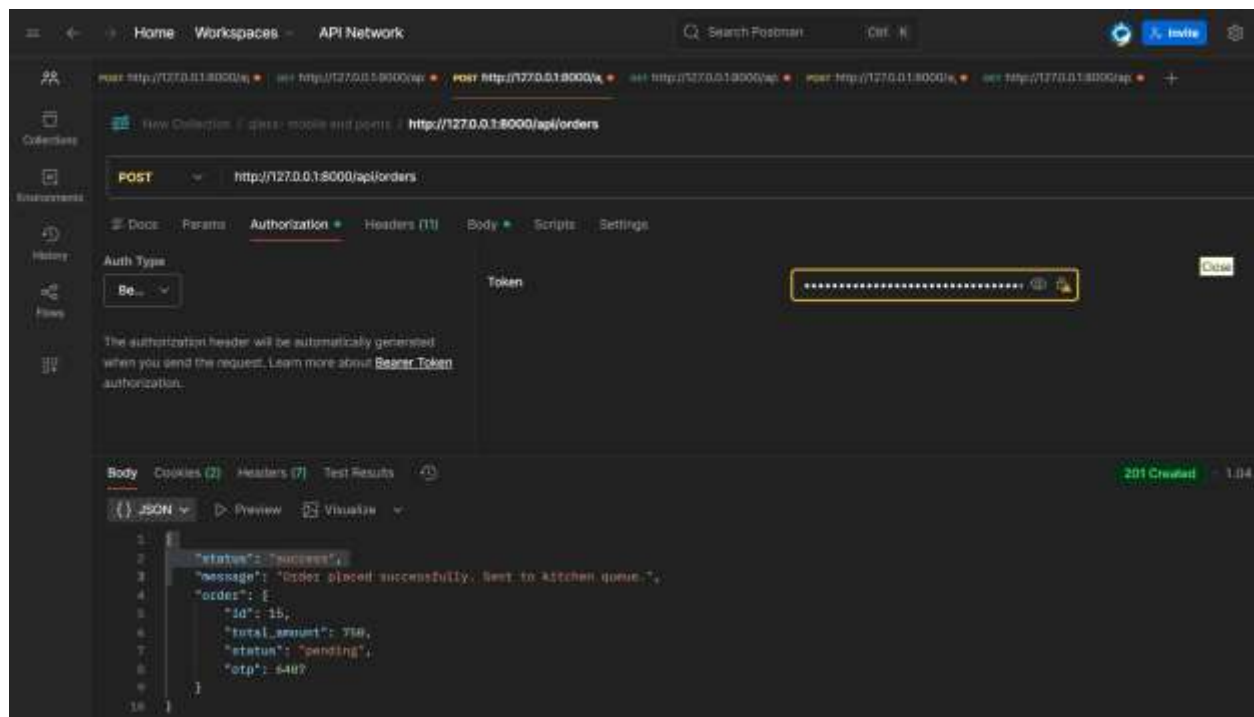


Figure 12: Postman verification valid multi-item cart sent with POST request to /api/orders returns a 201 Created status code confirming a successful database transaction and return a OTP.

3.4. Test Case 4: Log out form a sanctum session (POST /api/logout)

ID	Scenario	Test Steps	Input Data	Expected Output	Status
TC – 4.1	Logout form the session and try to hit the menu with that session token	1. POST to /api/logout 2.retry GET /api/menu	Headers: Accept: application/json Authorization: Blank Bearer <valid_token>	200 OK Status to logout. For menu 401 Unauthenticated.	PASS

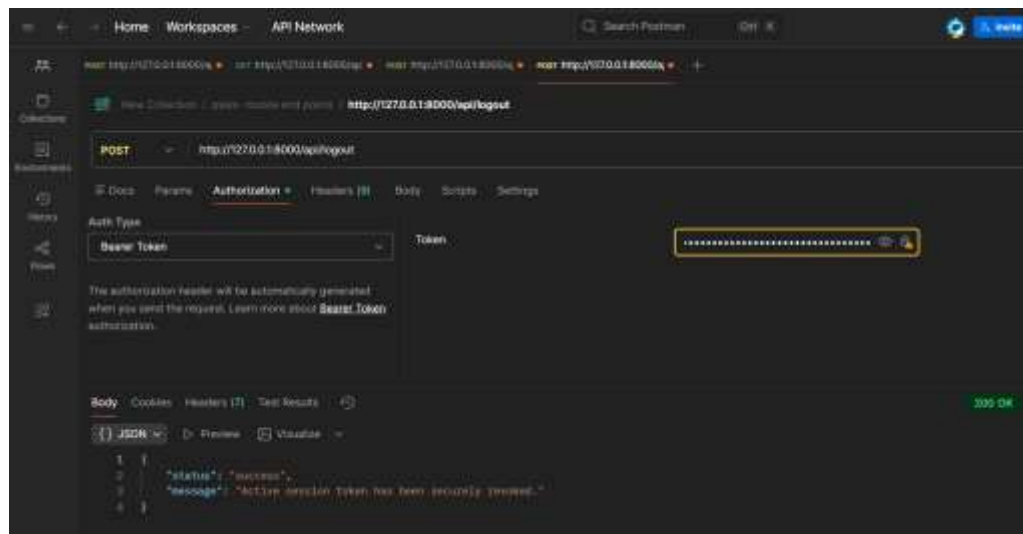


Figure 13: Logout Postman testing

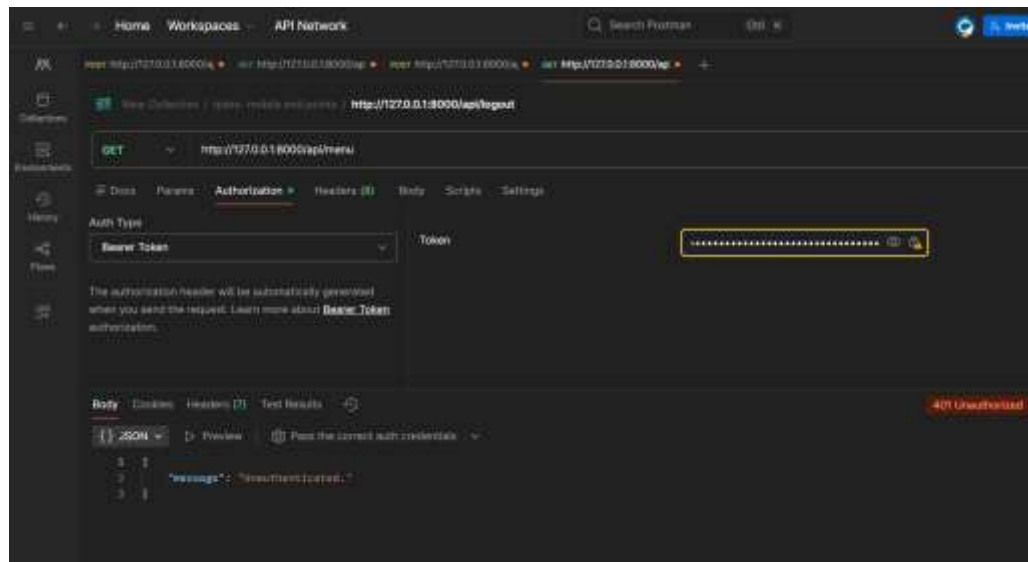


Figure 14: Postman Testing retry to hit menu end point with expired token

4 Hosting and Live Cloud Development

4.1 Production Infrastructure and asset pipeline

Live Website Link: <https://qlesscampus.infinityfreeapp.com/>

Qless Campus is hosted live on a free web hosting platform InfinityFree which uses customized cPanel control panel.

Fixing Asset Pipeline Error:

Because of the free hosting platform need all the web files to be in htdocs folder laravel's usual file structure had to be adjusted. For laravel to find the files custom path settings are added to the index.php to force Laravel to locate public assets correctly.

URL Rewriting:

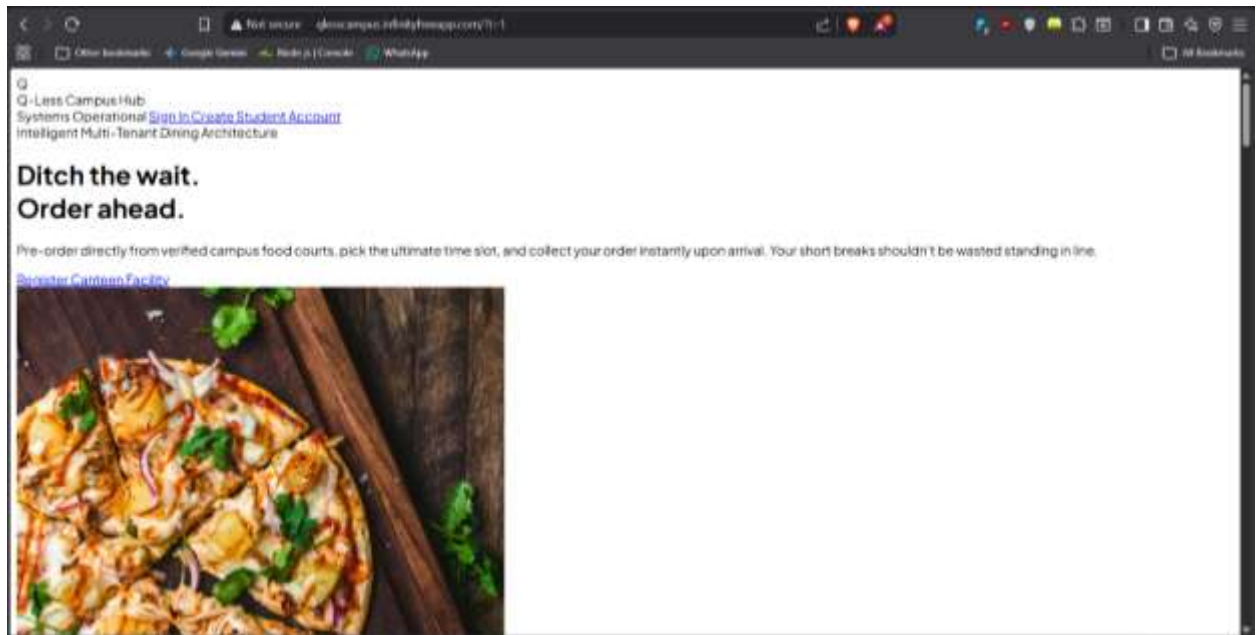


Figure 15: infinityFree's Security is blocking asset requests

Custom configuration is written in .htaccess file. Which tell the server to look in the public/build folder when the browser asks for CSS or Javascript to prevent layout and styles breaking (Figure 15: infinityFree's Security is blocking asset requests).

4.2 Database Management

Live Database:

Infinity Free use a cloud hosted MySQL database build inside a hosting platform. The database tables are migrated from local development computer using phpMyAdmin.

Production Setup (.env):

```
APP_ENV=production
APP_DEBUG=false
DB_CONNECTION=mysql
SESSION_DRIVER=cookie
```

Table of Figures

Figure 1: Web Registration Controller using Laravel Eloquent models to protect SQL Injection.	4
Figure 2: Mobile Authentication controller utilizing database parameters filtering to block SQL Injection	4
Figure 3: The hidden CSRF token code protecting for new food item creating from	5
Figure 4: Figure 4: Using laravel server side token check to stop access control leak - Menu.....	6
Figure 5: Using Laravel Server side token checks to stop access control leak – Order	6
Figure 6: Database safeguard for a clean order creating.....	7
Figure 7: Postman Verification showing successful request and return text Sanctum bearer token	9
Figure 8: Postman test showing 422 unprocessable content with automatic validation field error.	10
Figure 9: Postman test sending a wrong password returning a 401 Unauthenticated code	11
Figure 10: Postman testing (api/menu) with active bearer token	12
Figure 11: Postman Testing (/api/menu) without the Bearer token	13
Figure 12: Postman verification valid multi-item cart sent with POST request to /api/orders returns a 201 Created status code confirming a successful database transaction and return a OTP.	14
Figure 13: Logout Postman testing	15
Figure 14: Postman Testing retry to hit menu end point with expired token	15
Figure 15: infinityFree's Security is blocking asset requests	16